

Prueba de Caja Blanca

“Programa de inventario”

Integrantes

Adriana Astudillo

Sarahi Muñoz

Alan Nero

Fecha:

2025/07/25

Prueba caja blanca de Requisito N° 1: El programa debe permitir hacer el ingreso de datos necesarios para crear un producto nuevo, validar que el id sea único y confirmar que el producto se agregó exitosamente.

1. CÓDIGO FUENTE

```
//RF1: Crear producto con confirmación y Valida si el ID único
int idUnico(char id[]) {
    for (int i = 0; i < cantidad; i++) {
        if (strcmp(lista[i].id, id) == 0 && strlen(lista[i].id) > 0) {
            return 0;
        }
    }
    return 1;
}

void crearProducto(Producto *p) {
    // Validación del ID (no vacío y único)
    while (1) {
        printf("Ingrese ID: ");
        fgets(p->id, 20, stdin);
        p->id[strcspn(p->id, "\n")] = '\0'; // Eliminar salto de línea

        if (strlen(p->id) == 0) {
            printf(RED "Error: El ID no puede estar vacío.\n" RESET);
        } else if (!idUnico(p->id)) {
            printf(RED "Error: ID ya existe. Intente nuevamente.\n" RESET);
        } else {
            break; // ID válido
        }
    }
    confirmación y Valida si el ID único
}

int idUnico(char id[]) {
    for (int i = 0; i < cantidad; i++) {
        if (strcmp(lista[i].id, id) == 0 && strlen(lista[i].id) > 0) {
            return 0;
        }
    }
    return 1;
}

void crearProducto(Producto *p) {
    // Validación del ID (no vacío y único)
    while (1) {
        printf("Ingrese ID: ");
        fgets(p->id, 20, stdin);
        p->id[strcspn(p->id, "\n")] = '\0'; // Eliminar salto de línea

        if (strlen(p->id) == 0) {
            printf(RED "Error: El ID no puede estar vacío.\n" RESET);
        } else if (!idUnico(p->id)) {
            printf(RED "Error: ID ya existe. Intente nuevamente.\n" RESET);
        } else {
            break; // ID válido
        }
    }
}

// Validación del nombre del producto (no vacío)
while (1) {
    printf("Ingrese Producto: ");
```

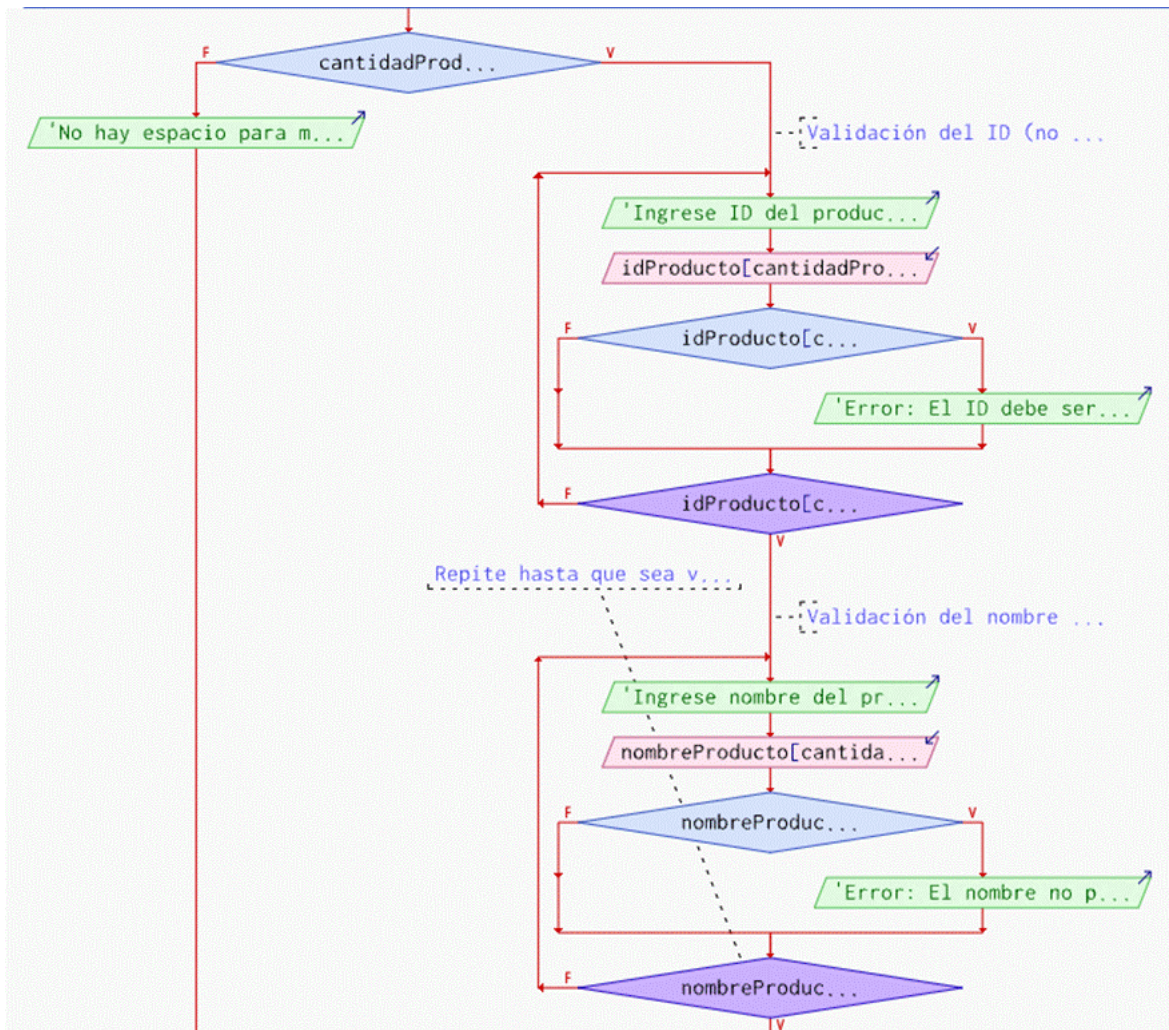
```

fgets(p->Producto, 50, stdin);
p->Producto[strcspn(p->Producto, "\n")] = '\0';

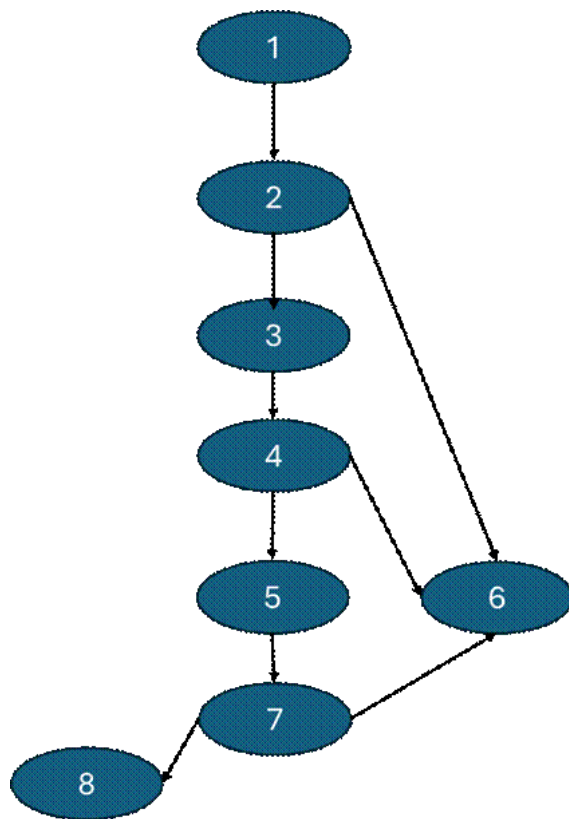
if (strlen(p->Producto) == 0) {
printf(RED "Error: El nombre del producto no puede estar vacío.\n" RESET);
} else {
break; // Nombre válido
}
}

```

2. DIAGRAMA DE FLUJO (DF) PSEINT



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1, 2, 3, 4, 5, 3, 4, 6, 7, 8, 6, 7, 9

R2: 1, 2, 3, 4, 6, 7, 8, 6, 9

R3: 1, 2, 3, 4, 5, 3, 6, 7, 9

R4: 1, 2, 3, 4, 6, 7, 9

R5: 1, 2, 9

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = 3 + 1 \quad V(G) = 4$
- $V(G) = 12 - 9 + 2 \quad V(G) = 5$

DONDE:

P: 3

A: 12

N: 9

Prueba caja blanca de Requisito N° 2: El programa deberá ingresar el producto al listado del inventario.

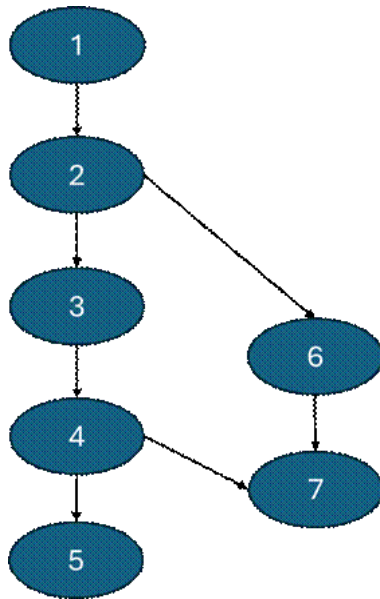
1. CÓDIGO FUENTE

```
//RF2: Guardar productos en archivo y Cargar productos desde archivo
void cargarDesdeArchivo() {
    crearCarpetaInventario(); // Asegurar que existe la carpeta
    FILE *archivo = fopen(ARCHIVO, "r");
    if (archivo == NULL) {
        printf(YELLOW "Creando nuevo archivo de inventario...\n" RESET);
        return;
    }
    while (!feof(archivo) && cantidad < MAX_PRODUCTOS) {
        if (fscanf(archivo, "%[^|]|%[^|]|%[^|]|%d\n",
            lista[cantidad].id,
            lista[cantidad].Producto,
            lista[cantidad].Marca,
            &lista[cantidad].Cantidad) == 4) {
            cantidad++;
        }
    }
    fclose(archivo);
}
```

2. DIAGRAMA DE FLUJO (DF) PSEINT



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1, 2, 4, 5, 6, 7, 4, 8, 9

R2: 1, 2, 4, 5, 7, 4, 8, 9

R3: 1, 2, 3, 9

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = 3 + 1 \quad V(G) = 4$
- $V(G) = 10 - 9 + 2 \quad V(G) = 3$

DONDE:

P: 3

A: 10

N: 9

Prueba caja blanca de Requisito N° 3: El programa deberá eliminar el producto del listado del inventario.

1. CÓDIGO FUENTE

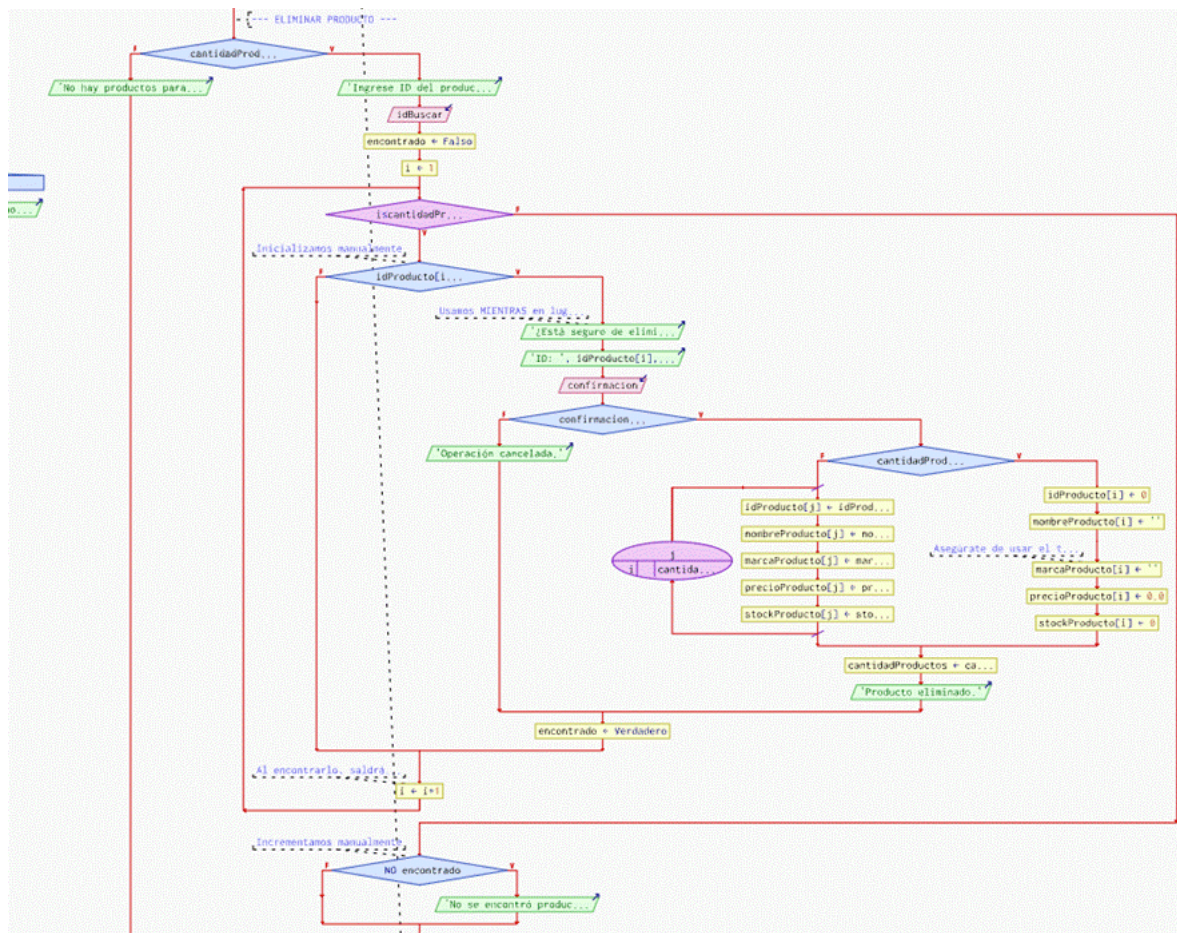
```
int posEliminar = buscarPorID(lista, cantidad, buffer);
if (posEliminar != -1 && strlen(lista[posEliminar].id) > 0) {
    printf(YELLOW "\n=== CONFIRMACION DE ELIMINACION ===\n"
RESET);
    printf(CYAN "Producto a eliminar:\n" RESET);
    mostrarProducto(lista[posEliminar]);
    printf(RED "\nADVERTENCIA:*Esta accion no se puede deshacer*\n"
RESET);
    printf(YELLOW "Escriba 'ELIMINAR-%%s'\n" RESET, buffer);

    char confirmacion[30];
    fgets(confirmacion, 30, stdin);
    confirmacion[strcspn(confirmacion, "\n")] = '\0';

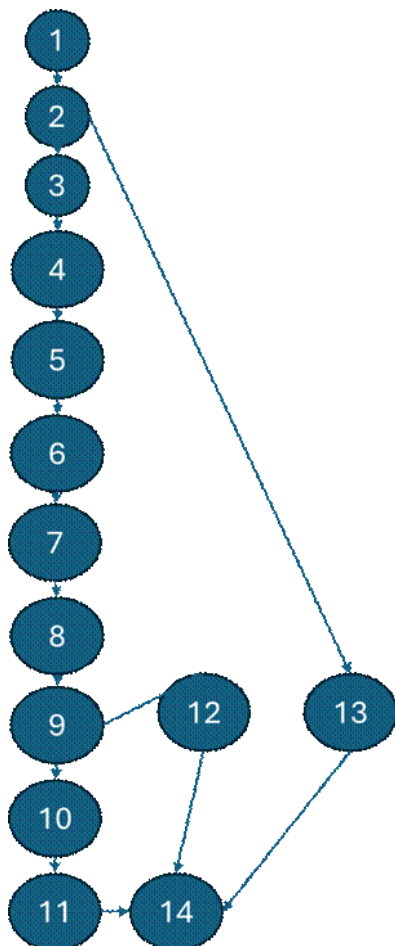
    char confirmacionEsperada[30];
    sprintf(confirmacionEsperada, "ELIMINAR-%%s", buffer);

    if (strcmp(confirmacion, confirmacionEsperada) == 0) {
        eliminarProducto(&lista[posEliminar]);
        guardarEnArchivo();
        printf(GREEN "\nProducto eliminado.\n" RESET);
    } else {
        printf(YELLOW "\nConfirmacion fallida. No se elimino.\n" RESET);
    }
} else {
    printf(RED "Producto no encontrado.\n" RESET);
}
break;
}
```

2. DIAGRAMA DE FLUJO (DF) PSEINT



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 14

R2: 1, 2, 3, 4, 5, 6, 7, 8, 9, 12, 14

R3: 1,2,13,14

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = 2 + 1 \quad V(G) = 3$
- $V(G) = 15 - 14 + 2 \quad V(G) = 3$

DONDE:

P: 2

A: 15

N: 14

Prueba caja blanca de Requisito N° 4: El programa deberá visualizar un listado en orden alfabético de todos los productos ingresados.

1. CÓDIGO FUENTE

// RF4: Ordenar productos por nombre (orden alfabético)

```
void ordenarPorNombre() {
    qsort(lista, cantidad, sizeof(Producto), compararProductos);
}
```

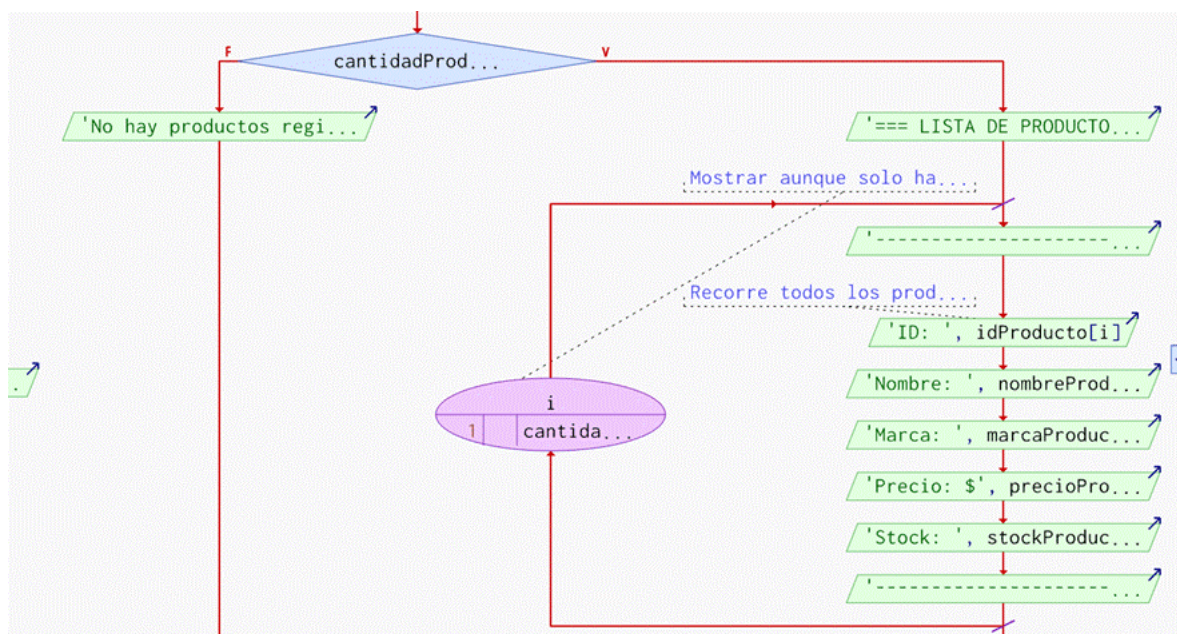
```
void mostrarTablaProductos() {
    ordenarPorNombre();
    printf("\n+-----+-----+-----+-----+\n");
    printf("| ID | PRODUCTO | MARCA | STOCK |\n");
    printf("+-----+-----+-----+-----+\n");
```

```
    int i = 0, alertas = 0, productosMostrados = 0;
    while (i < cantidad) {
```

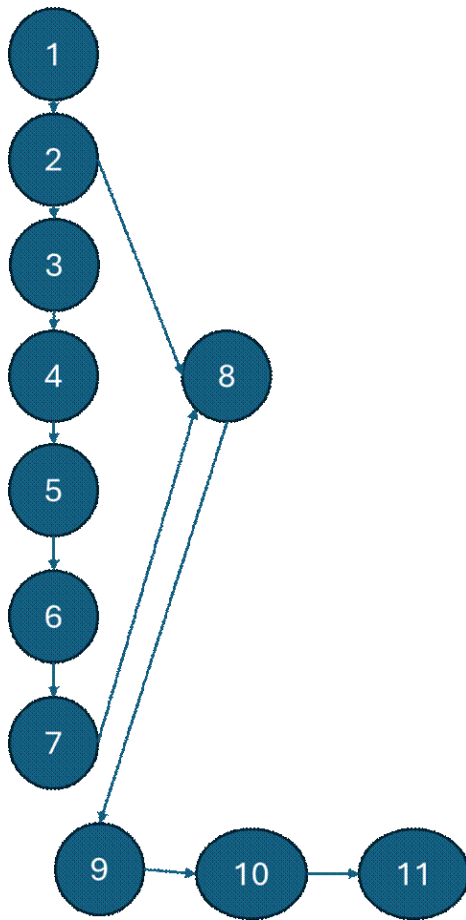
```
        printf("+-----+-----+-----+-----+\n");
        printf("\nTotal de productos registrados: %d\n", cantidad);
        if (alertas > 0) {
```

```
            printf(" " RED "*" Stock bajo o Producto agotado" RESET);
            printf(" * Stock normal\n");
        }
    }
```

2. DIAGRAMA DE FLUJO (DF) PSEINT



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1, 2, 3, 4, 5, 6, 7, 8, 9, 2, 10, 11, 12

R2: 1, 2, 3, 4, 6, 7, 8, 9, 2, 10, 11, 12

R3: 1, 2, 3, 9, 2, 10, 11, 12

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = 3 + 1 \quad V(G) = 4$
- $V(G) = 14 - 12 + 2 \quad V(G) = 4$

DONDE:

P: 3

A: 14

N: 12

Prueba caja blanca de Requisito N° 5: El programa deberá permitir la Búsqueda por ID/Nombre/Marca

1. CÓDIGO FUENTE

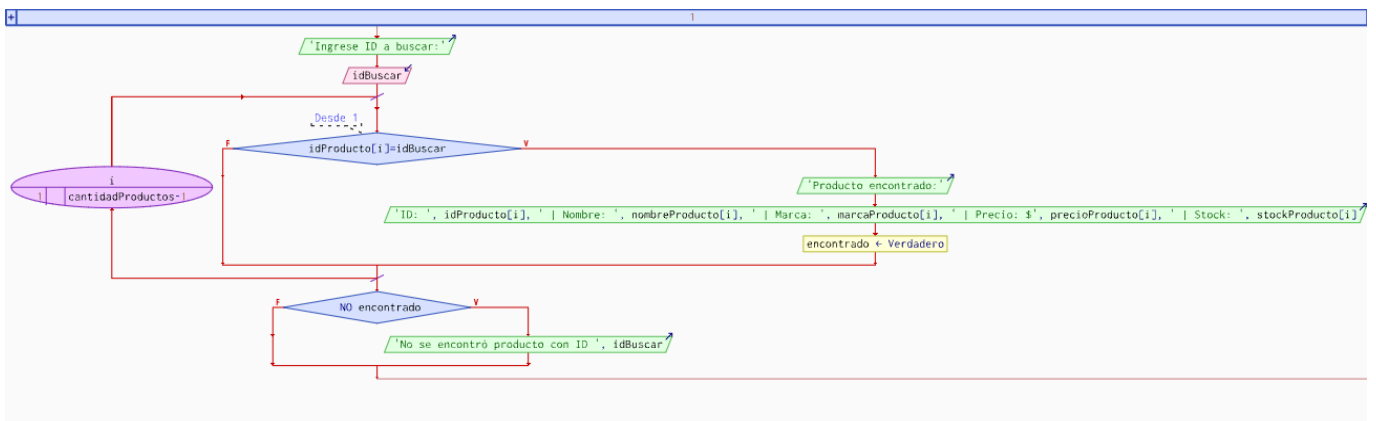
// RF5: Búsqueda por ID, nombre, marca

```
int buscarPorID(Producto lista[], int n, char id[]) {  
    int i = 0;  
    while (i < n) {  
        if (strcmp(lista[i].id, id) == 0) {  
            return i;  
        }  
        i++;  
    }  
    return -1;  
}
```

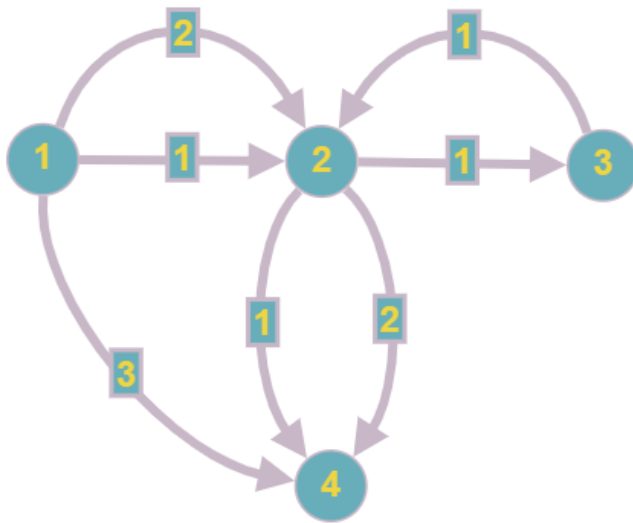
```
int buscarPorNombre(char nombre[]) {  
    int i = 0;  
    while (i < cantidad) {  
        if (strcmp(lista[i].Producto, nombre) == 0 && strlen(lista[i].id) > 0) {  
            return i;  
        }  
        i++;  
    }  
    return -1;  
}
```

```
int buscarPorMarca(char marca[]) {  
    int i = 0;  
    while (i < cantidad) {  
        if (strcmp(lista[i].Marca, marca) == 0 && strlen(lista[i].id) > 0) {  
            return i;  
        }  
        i++;  
    }  
    return -1;  
}
```

2. DIAGRAMA DE FLUJO (DF) PSEINT



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1,2,3,2,4

R2: 1,2,4

R3: 1, 4

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = A - N + 2$
- $V(G) = 5 - 4 + 2 = 3$
- $V(G) = P + 1$
- $V(G) = 2 + 1 = 3$

DONDE:

P: Número de nodos prediicado = 2

A: Número de aristas = 5

N: Número de nodos = 4

Prueba caja blanca de Requisito N° 6: El programa deberá presentar las existencias de los productos y alertar de la falta del stock.

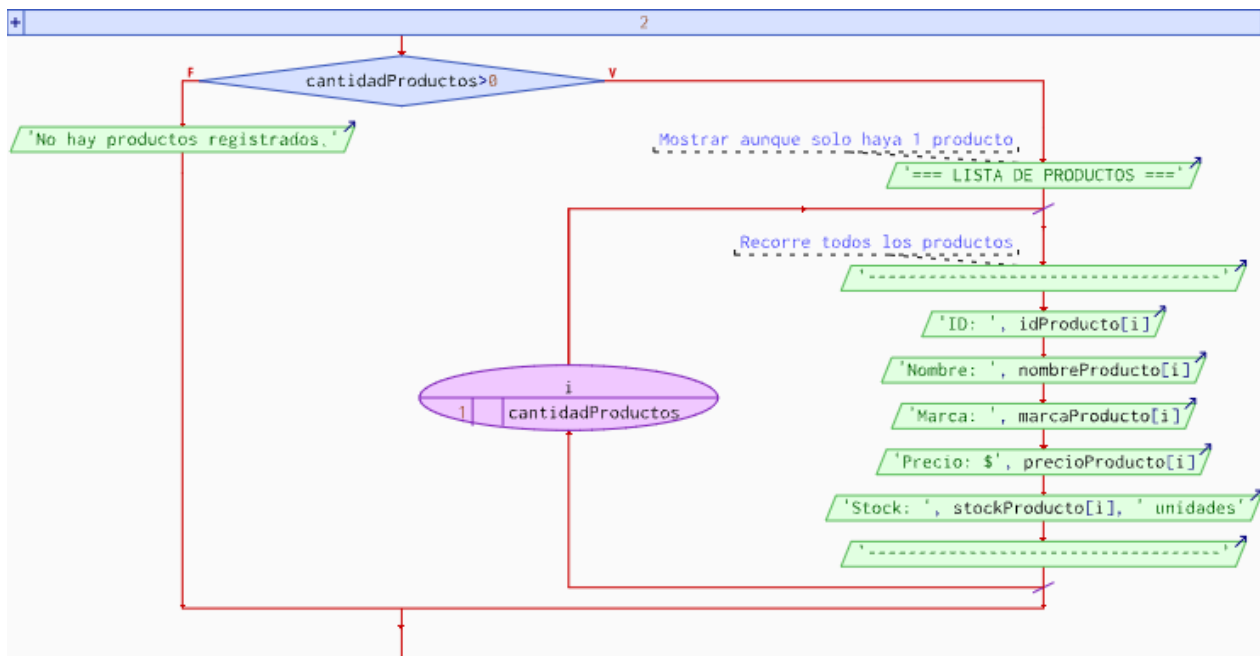
1. CÓDIGO FUENTE

//RF6: Mostrar producto con alerta de bajo stock

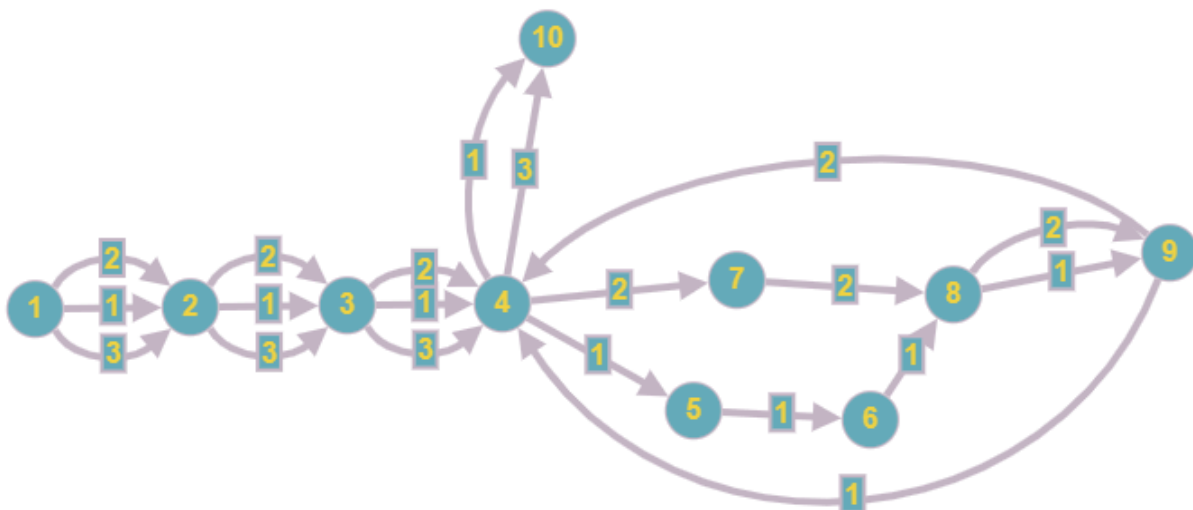
```
void mostrarProducto(Producto p) {
    printf("\nID: %s\n", p.id);
    printf("Producto: %s\n", p.Producto);
    printf("Marca: %s\n", p.Marca);
    printf("Cantidad: %d\n", p.Cantidad);

    if (p.Cantidad <= 5) {
        printf(RED "ALERTA: Stock bajo!\n" RESET);
    }
}
```

2. DIAGRAMA DE FLUJO (DF) PSEINT



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1,2,3,4,5,6,8,9,4,10

R2: 1,2,3,4,7,8,9,4,10

R3: 1,2,3,4,10

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = A - N + 2$
- $V(G) = 11 - 10 + 2 = 3$
- $V(G) = P + 1$
- $V(G) = 2 + 1 = 3$

DONDE:

P: Número de nodos prediado = 2

A: Número de aristas = 11

N: Número de nodos = 10

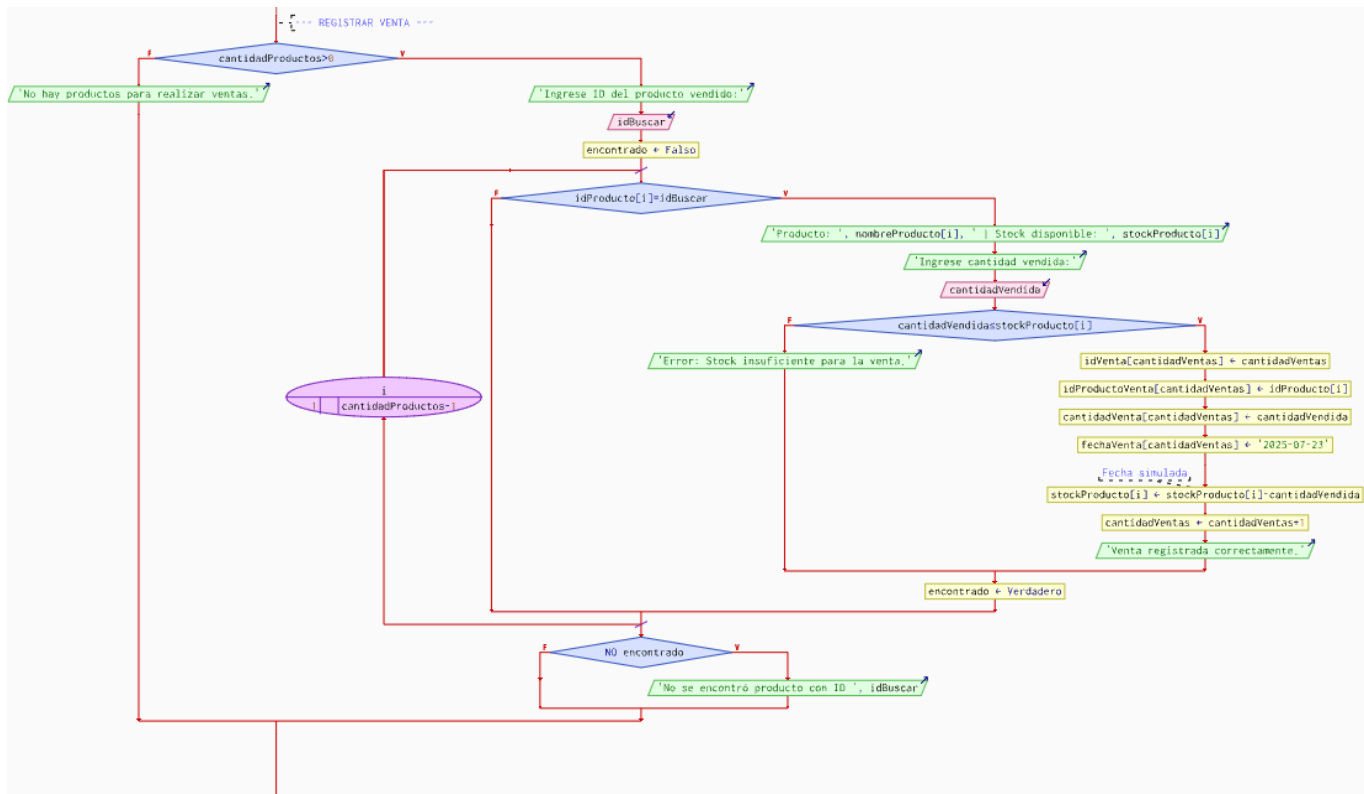
Prueba caja blanca de Requisito N° 7: El programa deberá presentar la cantidad de productos vendidos.

1. CÓDIGO FUENTE

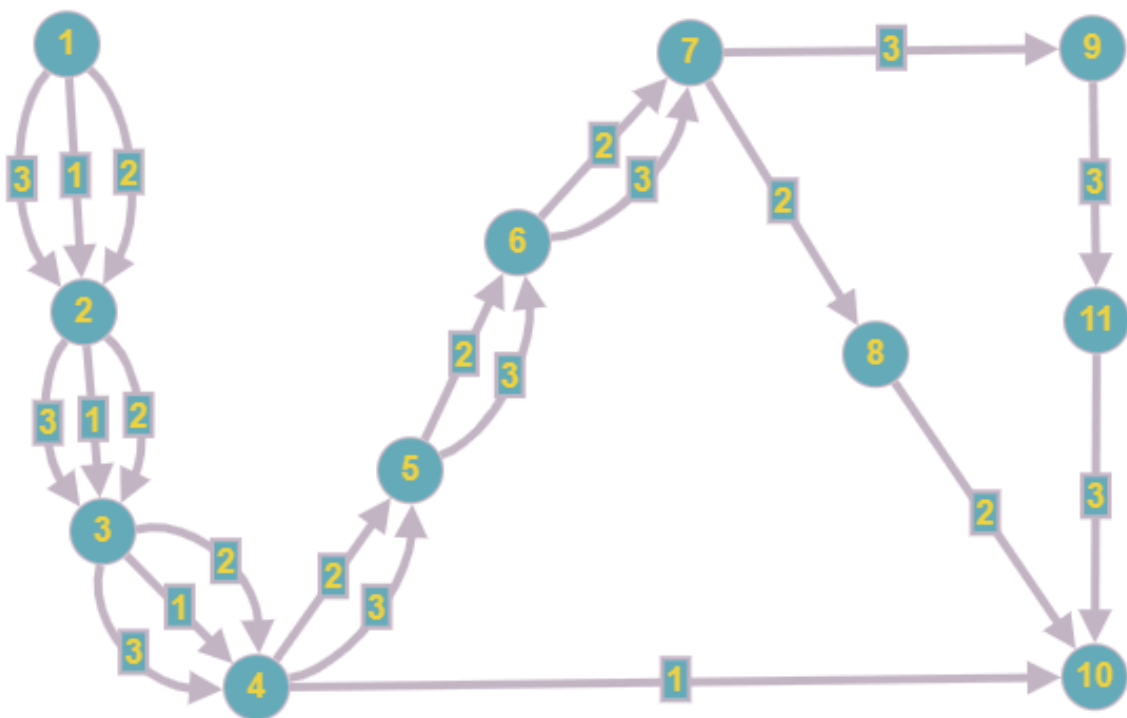
```
//RF7: Registrar ventas con validación de stock
void registrarVenta() {
    char id[20];
    printf("Ingrese ID del producto a vender: ");
    fgets(id, 20, stdin);
    id[strcspn(id, "\n")] = '\0';

    int pos = buscarPorID(lista, cantidad, id);
    if (pos == -1 || strlen(lista[pos].id) == 0) {
        printf(RED "\nError: Producto no encontrado.\n" RESET);
        return;
    }
    printf("Stock disponible: %d\n", lista[pos].Cantidad);
    int cantidadVendida;
    while (1) {
        printf("Ingrese cantidad a vender: ");
        if (scanf("%d", &cantidadVendida) != 1) {
            printf(RED "Error: Ingrese un número válido.\n" RESET);
            while (getchar() != '\n');
            continue;
        }
        getchar();
        if (cantidadVendida <= 0) {
            printf(RED "\nError: Cantidad no válida.\n" RESET);
        } else if (cantidadVendida > lista[pos].Cantidad) {
            printf(RED "\nError: Stock insuficiente.\n" RESET);
            printf("Intento vender: %d | Stock disponible: %d\n",
                cantidadVendida, lista[pos].Cantidad);
        } else {
            break;
        }
    }
    lista[pos].Cantidad -= cantidadVendida;
```

2. DIAGRAMA DE FLUJO (DF) PSEINT



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1,2,3,4,10

R2: 1,2,3,4,5,6,7,8,10

R3: 1,2,3,5,6,7,9,11,10

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = A - N + 2$
- $V(G) = 9 - 8 + 2 = 3$
- $V(G) = P + 1$
- $V(G) = 2 + 1 = 3$

DONDE:

P: Número de nodos predicado = 2

A: Número de aristas = 9

N: Número de nodos = 8

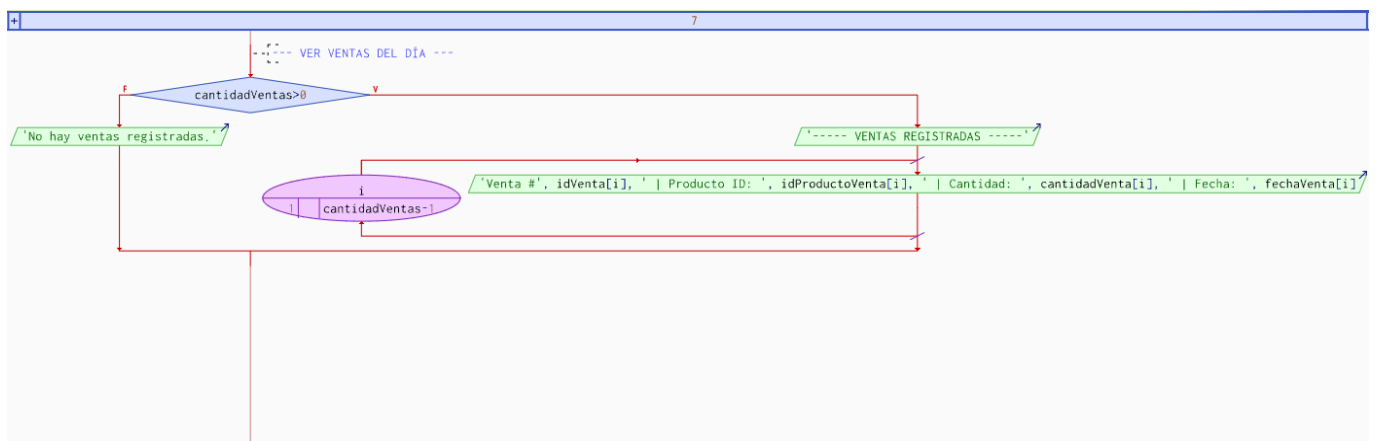
Prueba caja blanca de Requisito N° 8: El programa deberá dar a conocer el registro de ventas realizado en el día.

1. CÓDIGO FUENTE

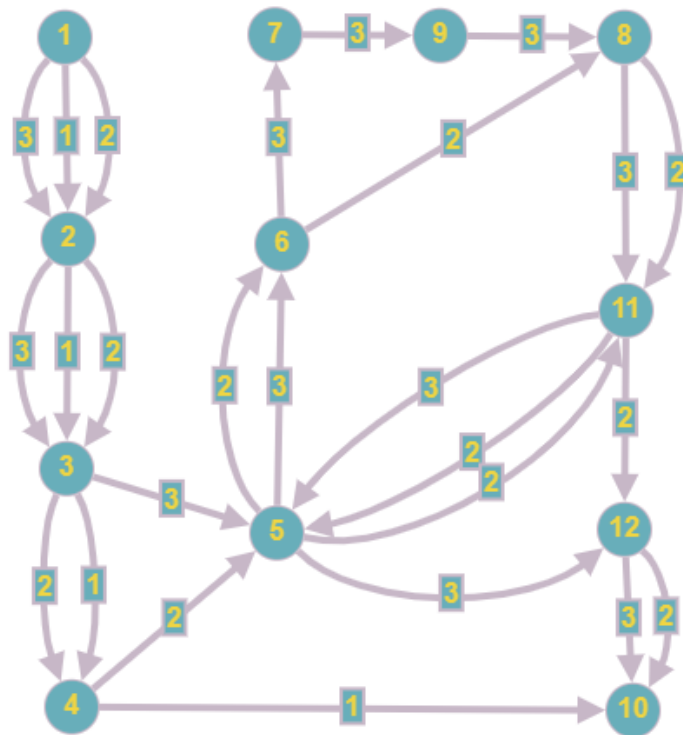
//RF8: Mostrar las ventas realizadas hoy y el total de productos vendidos

```
void mostrarVentasDelDia() {  
    FILE *ventas = fopen(ARCHIVO_VENTAS, "r");  
    if (ventas == NULL) {  
  
char fechaActual[11];  
    time_t t = time(NULL);  
    strftime(fechaActual, sizeof(fechaActual), "%Y-%m-%d", localtime(&t));  
  
    printf(GREEN "\nVentas realizadas hoy (%s):\n" RESET, fechaActual);  
    printf("-----\n");  
  
    char linea[200];  
    int totalVendidos = 0;  
  
    while (fgets(linea, sizeof(linea), ventas)) {  
        Venta v;  
        if (sscanf(linea, "%[^|]|%[^|]|%d|%^\\n", v.id, v.Producto, &v.CantidadVendida, v.fecha) == 4)  
        {  
            if (strcmp(v.fecha, fechaActual, 10) == 0) {  
                printf("Producto: %s | Cantidad: %d | Fecha: %s\\n",  
                    v.Producto, v.CantidadVendida, v.fecha);  
                totalVendidos += v.CantidadVendida;  
            }  
        }  
    }  
  
    printf("-----\n");  
    printf("Total productos vendidos hoy: %d\\n", totalVendidos);  
    fclose(ventas);  
}
```

2. DIAGRAMA DE FLUJO (DF) PSEINT



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1,2,3,4,10

R2: 1,2,3,4,5,6,8,11,5,11,12,10

R3: 1,2,3,5,6,7,9,8,11,5,12,10

R4: 1,2,3,5,6,7,9,8,11,12,10

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = A - N + 2$
- $V(G) = 11 - 9 + 2 = 4$
- $V(G) = P + 1$
- $V(G) = 3 + 1 = 4$

DONDE:

P: Número de nodos prediados = 3

A: Número de aristas = 11

N: Número de nodos = 9

Prueba caja blanca de Requisito N° 9: El programa deberá permitir editar los campos de los productos ingresados.

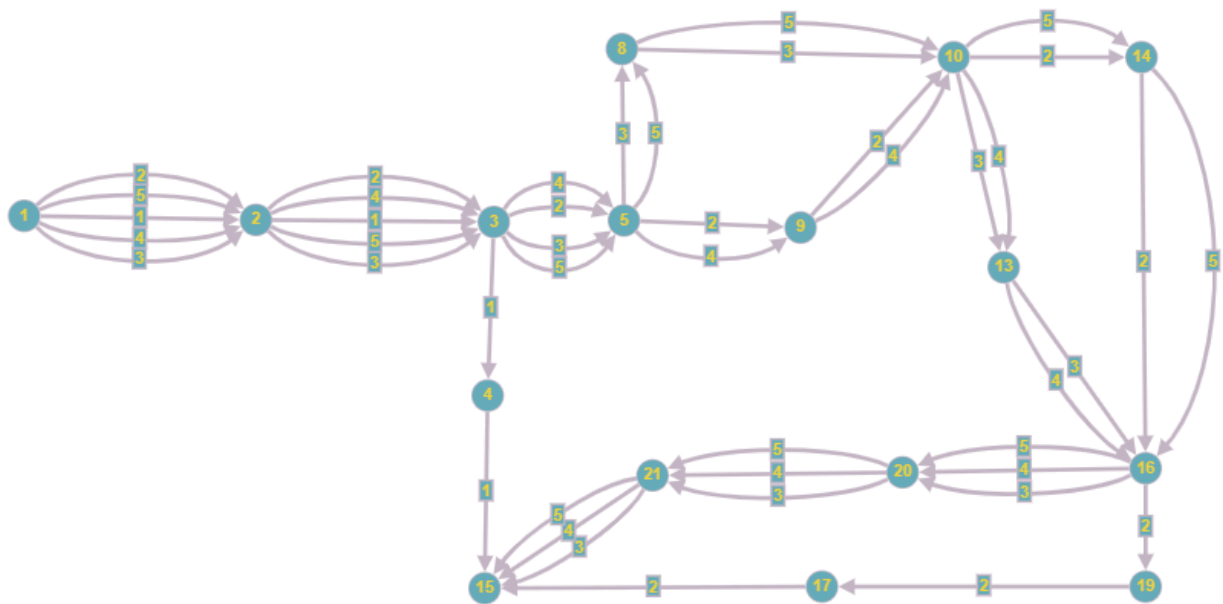
1. CÓDIGO FUENTE

```
//RF9: Editar producto existente
void editarProducto(char id[]) {
    int pos = buscarPorID(lista, cantidad, id);
    if (pos == -1) {
        printf(RED "Error: Producto no encontrado.\n" RESET);
        return;
    }
    printf(CYAN "\nEditando producto ID: %s\n" RESET, lista[pos].id);
    // Editar nombre
    while (1) {
        printf("Nombre actual: %s\nNuevo nombre (dejar vacio para mantener): ",
lista[pos].Producto);
        char nuevoNombre[50];
        fgets(nuevoNombre, 50, stdin);
        nuevoNombre[strcspn(nuevoNombre, "\n")] = '\0';
        if (strlen(nuevoNombre) > 0) {
            strcpy(lista[pos].Producto, nuevoNombre);
            break;
        } else {
            break;
        }
    }

    if (nuevaCantidad == -1) {
        break;
    } else if (nuevaCantidad >= 0) {
        lista[pos].Cantidad = nuevaCantidad;
        break;
    } else {
        printf(RED "Error: La cantidad no puede ser negativa.\n" RESET);
    }
}

guardarEnArchivo();
printf(GREEN "¡Producto actualizado!\n" RESET);
}
```


3.



R1: 1, 2, 3, 4, 15

R2: 1, 2, 3, 5, 9, 10, 14, 16, 19, 17, 15

R3: 1, 2, 3, 5, 8, 10, 13, 16, 20, 21, 15

R4: 1, 2, 3, 5, 9, 10, 13, 16, 20, 21, 15

R5: 1, 2, 3, 5, 8, 10, 14, 16, 20, 21, 15

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = A - N + 2$
- $V(G) = 18 - 15 + 2 = 5$
- $V(G) = P + 1$
- $V(G) = 5 + 1 = 6$

DONDE:

P: Número de nodos prediado = 5

A: Número de aristas = 18

N: Número de nodos = 15